Failure (an endless loop, and a real pause/resume) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `loop_limit_interrupt_practice.py`: this builds a graph that loops forever on purpose and proves a step counter actually stops it, then uses `interrupt()` to pause a graph mid-run and shows that a `MemorySaver` checkpoint is lost on restart while a `SqliteSaver` one survives. Watching an endless loop actually hit its limit, and a paused graph actually resume across a real process restart, is the difference between trusting these mechanisms and just believing the docs about them. **If not:** the Build Task's own checkpointer choice (`MemorySaver` vs. `SqliteSaver`) would be a guess instead of something you watched fail and then fixed yourself.

This exercise has 2 separate halves — the endless loop, and the checkpoint/interrupt/resume cycle. The solution below covers both.

Solution

Part 1:

```
# loop_limit_interrupt_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.errors import GraphRecursionError

class LoopState(TypedDict):
    count: int

def node_a(state: LoopState) -> dict:
    return {"count": state["count"] + 1}

def node_b(state: LoopState) -> dict:
    return {"count": state["count"] + 1}

builder = StateGraph(LoopState)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_a")
builder.add_edge("node_a", "node_b")
```

```
builder.add_edge("node_b", "node_a")

graph = builder.compile()

try:
    final_state = graph.invoke({"count": 0}, config={"recursion_limit": 5})
    # why: unreachable by design -- there's no exit edge from this loop
    print(f"Somehow finished: {final_state}")
except GraphRecursionError:
    print("Confirmed: an exit-less loop fails loudly, not silently.")
```

Expected output:

```
Confirmed: an exit-less loop fails loudly, not silently.
```

Part 2:

```
# loop_limit_interrupt_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Command

class ApprovalState(TypedDict):
    task: str
    approved: bool

def do_task(state: ApprovalState) -> dict:
    decision = interrupt({"question": f"Approve running: {state['task']}?"})
    return {"approved": decision}

builder = StateGraph(ApprovalState)
builder.add_node("do_task", do_task)
builder.add_edge(START, "do_task")
builder.add_edge("do_task", END)

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "run-1"}}
starting_state = {"task": "send an email", "approved": False}

paused_state = graph.invoke(starting_state, config=config)

pending_interrupts = paused_state.get("__interrupt__", [])
if pending_interrupts:
    question = pending_interrupts[0].value["question"]
```

```
    print(f"Graph paused, asking: {question}")
else:
    print("Graph did not pause -- something's wrong with the interrupt wiring")

resumed_state = graph.invoke(Command(resume=True), config=config)
print(f"Final state after resume: {resumed_state}")
```

Expected output:

```
Graph paused, asking: Approve running: send an email?
Final state after resume: {'task': 'send an email', 'approved': True}
```